Eigendecomposition

The directions a matrix can’t turn

Prof. Nipun Batra

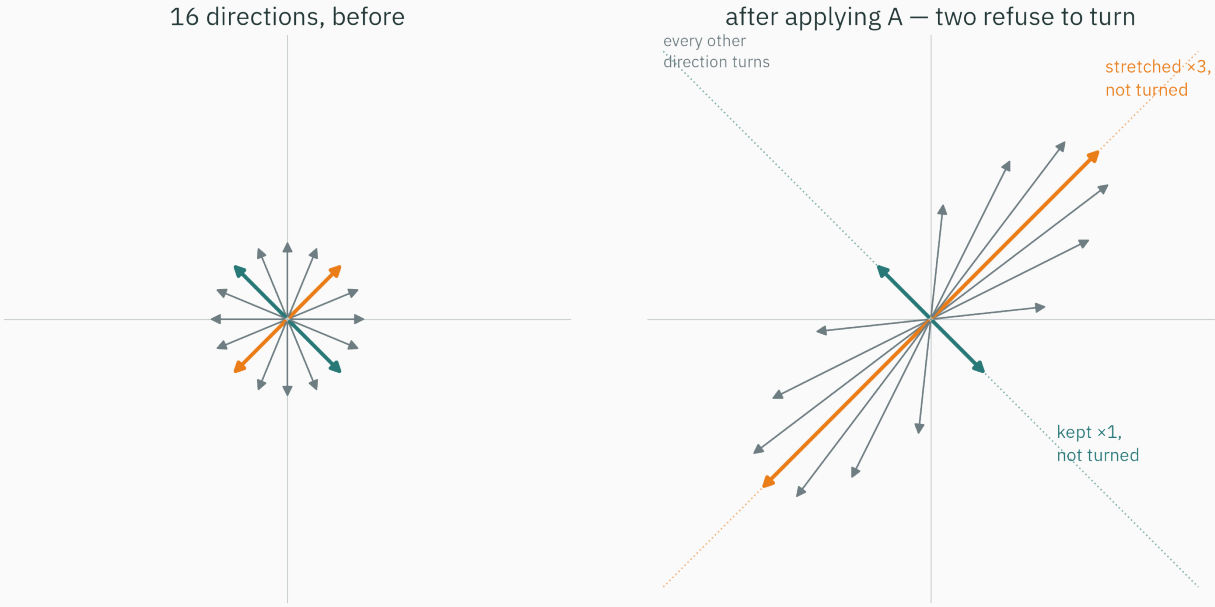
Mathematical Foundations for AI · IIT Gandhinagar

PageRank and repeated matrix multiplication

PageRank uses a stationary direction

V VISUAL

L1 demoed the teaching contract — picture, numbers, code, symbols — on **this exact picture**:



Almost every direction gets turned. Two refuse. Today we go deep: where they come from, what they cost to find, and why they quietly run modern AI.

1998 · the web has a ranking problem

Keyword-based ranking was easy to manipulate with repeated or hidden terms. Link structure offered a second signal: which pages point to this one?

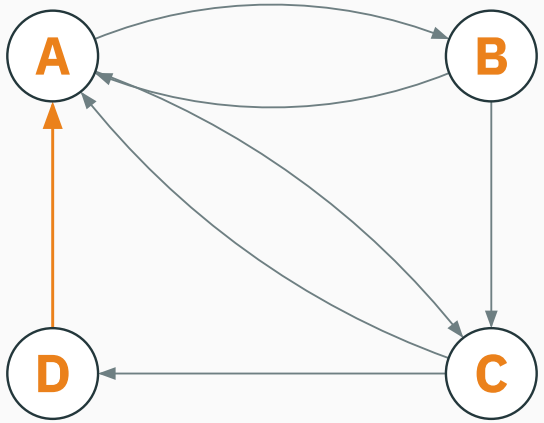
Two Stanford grad students, Sergey Brin and Larry Page, proposed something structural: ignore what a page *says about itself*; look at **who links to it**.

PageRank became a core component of early Google search. Its mathematical centre is an eigenvector of a link-transition matrix, which you will compute before this lecture ends.

A four-page internet

V VISUAL

Every arrow is a hyperlink:



Which page is the most important?

“Important” is circular — and that’s the good part

A link is a **vote**. First guess: count votes. But surely a vote *from an important page* should count for more…

- To score a page, you need the scores of the pages linking to it.
- To score *those*, you need the scores of the pages linking to **them**. And so on, forever.

The ranking must be self-consistent — a vector that reproduces itself under the voting rule. That is an eigenvector equation.

We’ll compute this ranking live today. The **full** model — dead ends, teleportation, why a unique answer is guaranteed — is **L25** (Markov chains), where PageRank is fully resolved.

Learning outcomes

By the end of this lecture you will be able to:

1. Say what an **eigenvector** and **eigenvalue** are — in pictures, numbers, code, and symbols.
2. Compute both **by hand** for any 2×2 matrix (characteristic polynomial).
3. Interpret a real λ as scaling and possible sign reversal; recognize that complex-conjugate pairs describe two-dimensional rotation–scaling behaviour.
4. Factor $A = PDP^{-1}$ and use it to compute A^k instantly.
5. Run **power iteration** — and prove why it finds the top eigenvector.
6. State the **spectral theorem** for symmetric matrices (and see why ML loves them).
7. Explain the eigen-idea inside **PageRank**.

Directions that don’t turn

Numbers first: catch a direction refusing to turn

Same matrix as L1. Feed it $\boldsymbol{v} = (1, 1)$:

$$A\boldsymbol{v} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 2 \cdot 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix} = 3\boldsymbol{v}$$

Output = **3 × input**. Same line through the origin — just stretched. Now feed it an ordinary vector, $\boldsymbol{u} = (1, 0)$:

$$A\boldsymbol{u} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$ — was pointing east, now points east-north-east: **turned**.

The second stubborn direction

Try $w = (1, -1)$:

$$Aw = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 - 1 \\ 1 - 2 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} = 1 \cdot w$$

Not turned — not even stretched. Stretch factor exactly 1.

A has two special directions: $(1, 1)$ with factor 3, and $(1, -1)$ with factor 1. Everything this lecture builds runs on those two facts.

Code: NumPy finds both in one line

```
>>> A = np.array([[2., 1.], [1., 2.]])
>>> vals, vecs = np.linalg.eig(A)
>>> vals
array([3., 1.])
>>> vecs                          # eigenvectors are the COLUMNS
array([[ 0.70710678, -0.70710678],
       [ 0.70710678,  0.70710678]])
```

Column k pairs with `vals[k]`. Column 0 is $(1, 1)$ normalized to unit length.

Column 1 is $(-0.707, 0.707)$ — that is $(1, -1)$ times $-1/\sqrt{2}$. Same line, so same eigenvector: NumPy just picked a different scale and sign.

Symbols, at last: the definition

A **nonzero** vector $\boldsymbol{v}$ is an **eigenvector** of a square matrix A, with **eigenvalue** λ, if

$$A\boldsymbol{v} = \lambda\boldsymbol{v}$$

- applying A to $\boldsymbol{v}$ only **scales** it by λ — it never leaves its line;
- *eigen* is German for “own / characteristic”: these are A’s **own** directions.

An eigenvector is a direction A cannot turn.
The eigenvalue is what A does instead of turning it.

The rules of the club

- **$v = 0$ is banned.** $A \cdot 0 = \lambda \cdot 0$ holds for every λ — no information.
- **Any nonzero multiple of v is the *same* eigenvector.** If $A\boldsymbol{v} = \lambda\boldsymbol{v}$, then $A(c\boldsymbol{v}) = \lambda(c\boldsymbol{v})$. Eigenvectors are **directions**, not arrows of a fixed length — we normalize purely for convenience.
- **$\lambda = 0$ is allowed!** $A\boldsymbol{v} = 0$ means $\boldsymbol{v}$ is squashed to the origin — eigenvalue-zero directions are exactly the **null space** from L4.
- λ may be negative, repeated, or (for some matrices) not real at all — all three coming up.

Interactive: drag a vector until it stops turning

I INTERACTIVE

INTERACTIVE

↗ setosa.io/ev/eigenvectors-and-eigenvalues/

Drag v around the plane and watch Av follow. When the two arrows line up, you have found an eigen-direction — feel how the plane snaps along them. Then edit the matrix and watch the special directions move.

Five minutes with this builds the reflex the algebra will lean on: **eigenvectors are found by watching directions, not by staring at matrix entries.**

Why two stubborn directions run everything

Apply A to $v = (1, 1)$ **twice**:

$$A(Av) = A(3v) = 3 \cdot Av = 9v \qquad A^{10}v = 3^{10}v = 59049\, v$$

On its eigen-directions, the matrix A^{10} — a hundred multiplications and additions — collapses to **one scalar power**.

Repeated application of a matrix is governed entirely by its eigenvectors. That single sentence is the spine of this lecture — and of PageRank.

Finding eigenvalues by hand

Turn the definition into something solvable

$A\boldsymbol{v} = \lambda\boldsymbol{v}$ has two unknowns tangled together: the direction $\boldsymbol{v}$ and the number λ. Move everything left:

$$A\boldsymbol{v} - \lambda\boldsymbol{v} = 0 \quad \Rightarrow \quad (A - \lambda I)\boldsymbol{v} = 0$$

The identity I sneaks in because $A - \lambda$ (matrix minus number) is not defined — λI scales every direction by λ, so subtracting it is legal and changes nothing else.

So we need: a **nonzero** $\boldsymbol{v}$ that the matrix $(A - \lambda I)$ sends to zero.

Nonzero-to-zero demands a collapse

L4 fact: an **invertible** matrix sends only 0 to 0 — every other input survives.

So if $(A - \lambda I)\boldsymbol{v} = 0$ has a nonzero solution, then $(A - \lambda I)$ must be **singular** — it squashes some direction flat. And L4 gave us the detector for that:

$$\det(A - \lambda I) = 0$$

the characteristic equation — solve it for λ, and the collapsed direction is the eigenvector

Most λ values leave $A - \lambda I$ invertible. Eigenvalues are the handful of **exact settings** at which it collapses.

Worked by hand: the polynomial

For our A, subtract λ down the diagonal and take the 2×2 determinant:

$$\det\begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)(2-\lambda) - 1 \cdot 1$$

$$= \lambda^2 - 4\lambda + 4 - 1 = \lambda^2 - 4\lambda + 3$$

A quadratic in λ — the **characteristic polynomial**. A 2×2 always gives a quadratic: **two** eigenvalues (counted with repeats).

Worked by hand: the roots

$$\lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1) = 0 \quad \Rightarrow \quad \lambda_1 = 3, \quad \lambda_2 = 1$$

Exactly the two stretch factors we caught by hand — and exactly NumPy's `array([3., 1.])`.

Honesty about scale: we solve the polynomial for 2×2 only. For a 1000×1000 matrix nobody touches a degree-1000 polynomial — not you, and not NumPy either (it uses iterative methods; you'll build the simplest one today).

Back-substitute $\lambda = 3$: recover the direction

Set $\lambda = 3$ in $(A - \lambda I)\boldsymbol{v} = 0$:

$$(A - 3I)\boldsymbol{v} = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \Rightarrow \quad v_2 = v_1$$

Both rows say the same thing — the matrix has **collapsed to one honest equation**, exactly as engineered. Pick $v_1 = 1$:

$$\boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{(or any multiple — it's a direction)}$$

Back-substitute $\lambda = 1$

$$(A - I)\boldsymbol{w} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \Rightarrow \quad w_2 = -w_1 \quad \Rightarrow \quad \boldsymbol{w} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Look at those two matrices: at $\lambda = 3$ and $\lambda = 1$ the rows became **copies of each other** — rank collapsed from 2 to 1 (L4′s rank, on duty). At any other λ, the only solution would be $\boldsymbol{v} = 0$.

2×2 recipe: $\det(A - \lambda I) = 0$ **→ quadratic → two λ's → back-substitute each → two directions.**

Two instant sanity checks (2×2 gold)

For any 2×2 (in fact any $n \times n$) matrix:

$\operatorname{trace}(A) = a_{11} + a_{22} = \lambda_1 + \lambda_2 \qquad \det(A) = \lambda_1 \lambda_2$

Our A: trace $= 2 + 2 = 4 = 3 + 1$ ✓ det $= 4 - 1 = 3 = 3 \cdot 1$ ✓

Ten-second exam armor: after **any** eigenvalue computation, check the sum against the trace and the product against the determinant. Catches most slips.

Checkpoint: read the eigenvalues

Q QUESTION

$M = \begin{pmatrix} 5 & 2 \\ 0 & 3 \end{pmatrix}$. **Its eigenvalues are…?** (try to answer **without** fully expanding the polynomial)

A. 5 and 3

B. 5 and 2

C. 2 and 3

D. 4 and 4

Answer: read the eigenvalues

A ANSWER

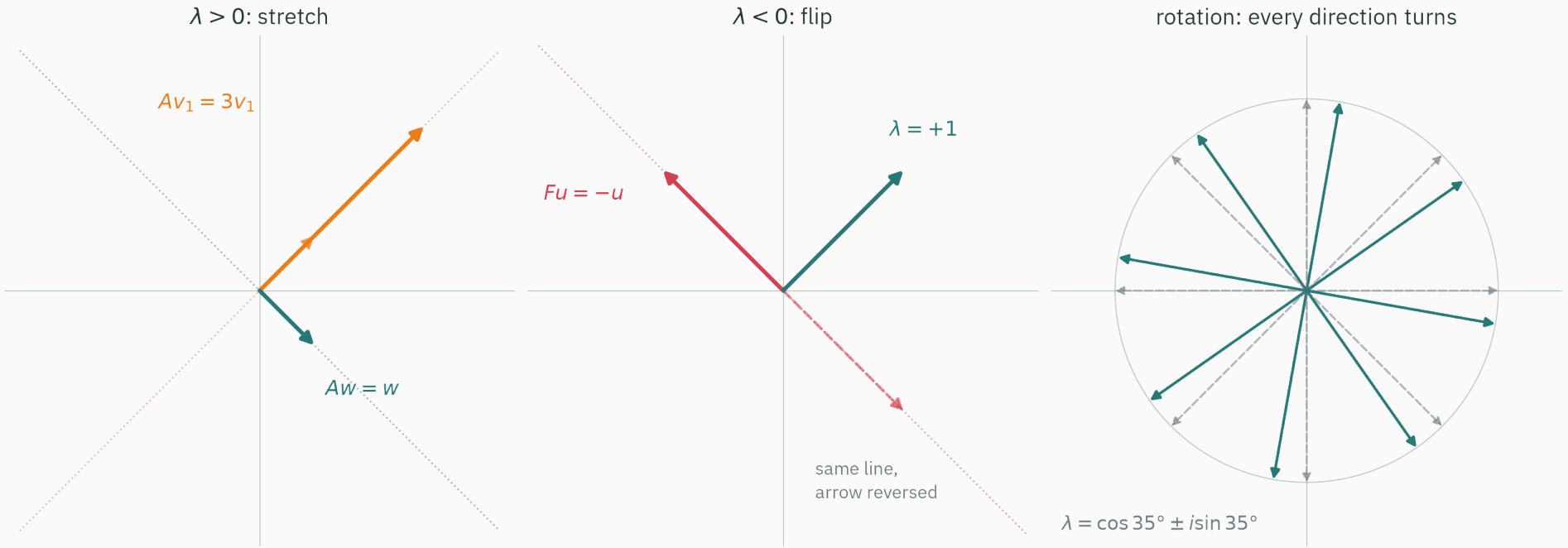
Correct: A — $\lambda = 5$ and $\lambda = 3$

Why: $\det(M - \lambda I) = (5 - \lambda)(3 - \lambda) - 2 \cdot 0$: the off-diagonal term dies, so for a **triangular** matrix the eigenvalues sit on the diagonal. Checks: trace $8 = 5 + 3$ ✓, det $15 = 5 \cdot 3$ ✓. (Careful: this shortcut is for triangular matrices only — our $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ has eigenvalues $3, 1$, not $2, 2$.)

Interpreting eigenvalues

One number per direction — what can it say?

V VISUAL



Three matrices, three behaviors: stretch along the eigen-line ($\lambda > 0$), **reverse** along it ($\lambda < 0$), or refuse to have a real eigen-line at all (rotation).

$\lambda < 0$: the direction that flips

The mirror matrix $F = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ swaps coordinates — reflection across $y = x$. Check $\boldsymbol{u} = (1, -1)$:

$$F\boldsymbol{u} = \begin{pmatrix} -1 \\ 1 \end{pmatrix} = -1 \cdot \boldsymbol{u}$$

$F\boldsymbol{u}$ lies on the **same line** as $\boldsymbol{u}$ — no turning! — but points the opposite way. That is $\lambda = -1$: kept in line, reversed in sense.

And $(1, 1)$? Reflection leaves it alone: $\lambda = +1$. Trace $0 = 1 + (-1)$ ✓, det $-1 = 1 \cdot (-1)$ ✓. A negative **determinant** always betrays a flip hiding somewhere (L4).

Rotation: the matrix with no real eigenvectors

Rotate every vector by $90°$: $R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. **Every** direction turns — so no eigenvector should exist. The algebra agrees:

$$\det(R - \lambda I) = \lambda^2 + 1 = 0 \quad \Rightarrow \quad \lambda = \pm i$$

No real roots. The eigenvalues have become a **complex pair** — and that is precisely the algebra's way of saying "this map rotates".

We will not use complex arithmetic beyond this slide: a complex eigenvalue pair represents a rotation, possibly combined with scaling. We will see the same effect once more today as a wobble in a live computation. A formal treatment of complex vector spaces is beyond this course.

Eigenvalue cases in one table

Eigenvalue	What happens along that direction
$\lambda > 1$	stretched by λ
$\lambda = 1$	untouched
$0 < \lambda < 1$	shrunk toward the origin
$\lambda = 0$	squashed flat — the null-space direction (L4)
$-1 \leq \lambda < 0$	flipped, and kept-or-shrunk
$\lambda < -1$	flipped **and** stretched
complex pair	no invariant direction — a rotation (+ scale) in that plane

Eigen-directions turn a matrix into a dashboard of dials — one independent stretch factor per direction.

Diagonalization: the eigen-basis

Package the eigen-data as matrices

Two facts, $A\boldsymbol{v} = 3\boldsymbol{v}$ and $A\boldsymbol{w} = 1\boldsymbol{w}$, stack into one equation. Put the eigenvectors in the **columns** of P, the eigenvalues in a diagonal D:

$$P = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \qquad D = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} \qquad \text{then} \quad AP = PD$$

(AP hits each column: $(A\boldsymbol{v}, A\boldsymbol{w}) = (3\boldsymbol{v}, 1\boldsymbol{w})$ — which is exactly PD.)

P's columns are independent directions, so P is invertible (L4). Multiply on the right by P^{-1}:

$A = PDP^{-1}$ — the eigendecomposition (a.k.a. diagonalization).

The slide computes P^{-1}, then the triple product, at compile time (chalkdust linalg):

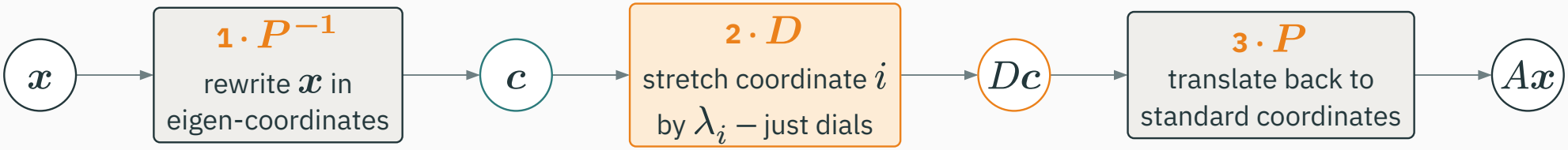
$$P^{-1} = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{pmatrix}$$

$$PDP^{-1} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \quad A \checkmark$$

computed, not typed — change P or D in the source and this slide would catch the lie

Read it right to left: three moves

$A\boldsymbol{x} = PDP^{-1}\boldsymbol{x}$ is a **recipe**:



In the right coordinate system, A **is** just its dials.

This is a change of basis (L3, paying off)

The eigenvectors $\{\boldsymbol{v}, \boldsymbol{w}\}$ are two independent directions — a perfectly good **basis** for the plane (L3). Any $\boldsymbol{x}$ has coordinates in it:

$$\boldsymbol{x} = c_1\boldsymbol{v} + c_2\boldsymbol{w}, \qquad \boldsymbol{c} = P^{-1}\boldsymbol{x}$$

In eigen-coordinates the tangled map $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ becomes the **diagonal** map $\begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}$: coordinate 1 tripled, coordinate 2 kept. No crosstalk between coordinates, ever.

Diagonalization = switching to the basis A itself prefers. Same map, honest coordinates.

Matrix powers in an eigenbasis

Multiply the factored form by itself and watch the middle cancel:

$$A^2 = PD\underbrace{P^{-1}P}_{=I}DP^{-1} = PD^2P^{-1}$$

Every extra factor of A cancels the same way, $k-1$ times:

$$A^k = PD^kP^{-1}, \qquad D^k = \begin{pmatrix} 3^k & 0 \\ 0 & 1^k \end{pmatrix} \quad \text{— powers of a diagonal matrix are free}$$

k matrix multiplications become two scalar powers and one change of basis.

A^{10}, two ways — both computed live

Way 1 (brute force): the slide multiplies A by itself ten times:

$$A^{10} = \begin{pmatrix} 29525 & 29524 \\ 29524 & 29525 \end{pmatrix}$$

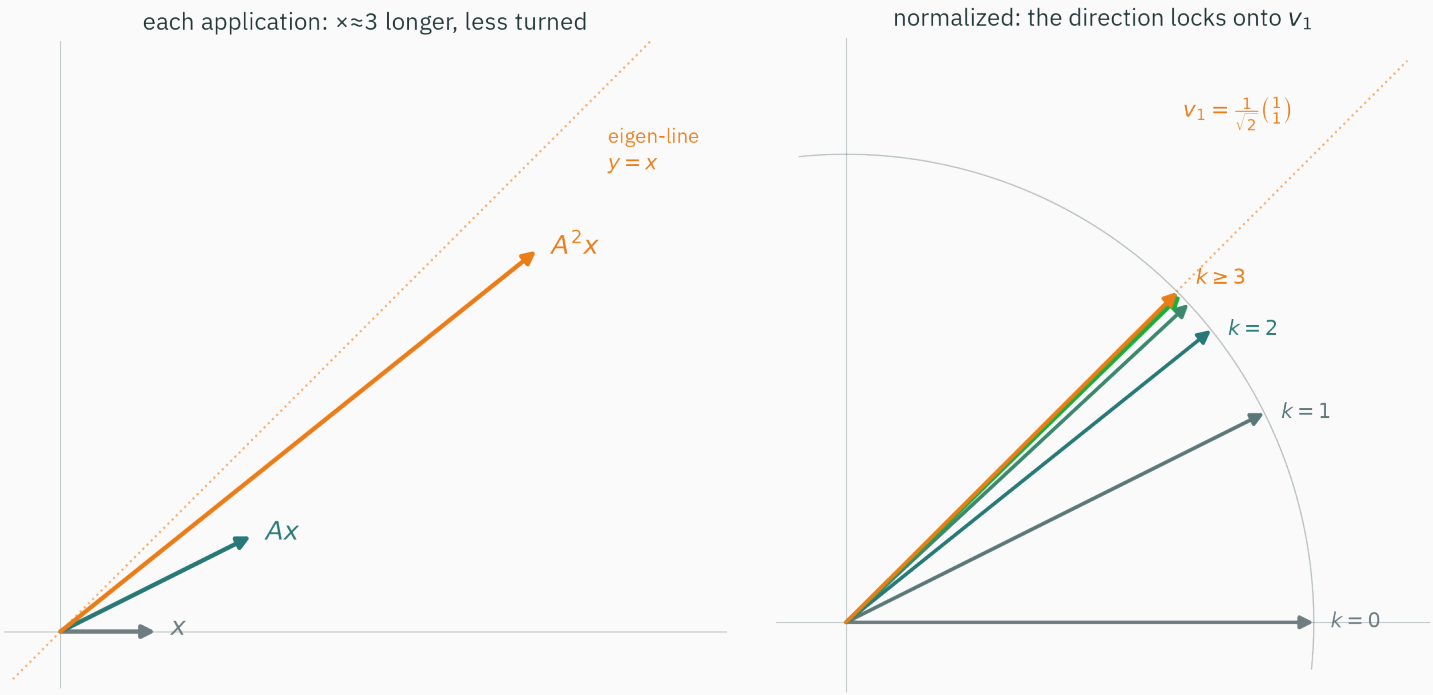
Way 2 (eigen): the slide computes $PD^{10}P^{-1}$ with $3^{10} = 59049$:

$$P\begin{pmatrix} 3^{10} & 0 \\ 0 & 1 \end{pmatrix}P^{-1} = \begin{pmatrix} 29525 & 29524 \\ 29524 & 29525 \end{pmatrix} \quad \text{same matrix } \checkmark$$

Every entry is $\approx 3^{10}/2$: after ten applications, the $\lambda = 3$ direction has utterly buried the $\lambda = 1$ direction.

Watch a vector under repeated application

V VISUAL



for a generic start, the component along the dominant eigenvector eventually determines the direction of $A^k x$

That observation gives a practical algorithm.

Power iteration, live

The algorithm: multiply, normalize, repeat

To find the top eigen-direction of A — **without** polynomials:

$$\boldsymbol{x}_{k+1} = \frac{A\boldsymbol{x}_k}{\|A\boldsymbol{x}_k\|} \qquad \text{start from (almost) any } \boldsymbol{x}_0$$

- **Multiply:** let A drag the direction wherever it wants.
- **Normalize:** rescale to unit length. The lengths would grow like 3^k — in float32 that overflows around $k \approx 81$, and you know exactly why (L2). Normalizing keeps only what matters: the **direction**.

This is power iteration — the simplest eigenvalue algorithm in existence, and the one that ranked the web.

Eight iterations, computed inside this slide

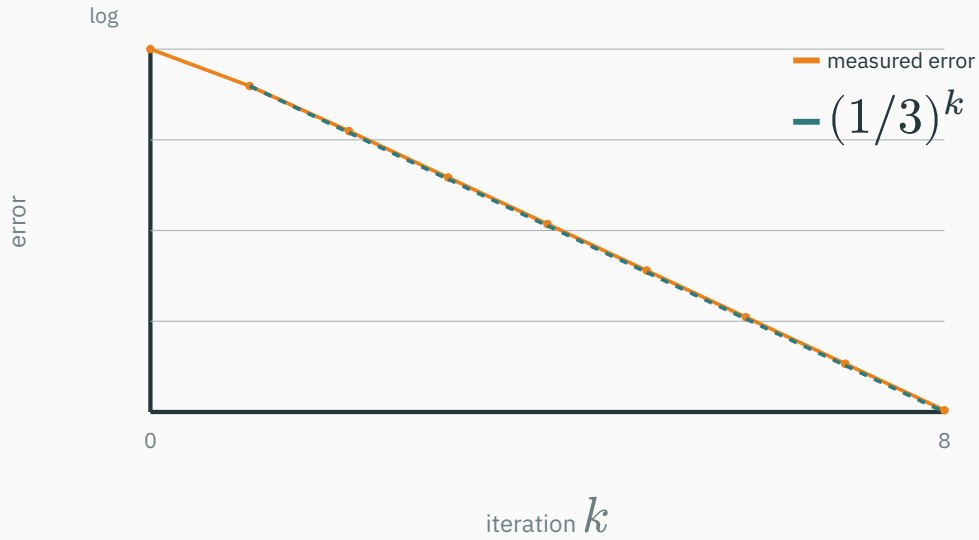
Start at $x_0 = (1, 0)$ — pointing due east, $45°$ away from the answer:

k	first coord of x_k	second coord	error $\|x_k - v_1\|$
0	1.0000	0.0000	0.76537
1	0.8944	0.4472	0.32036
2	0.7809	0.6247	0.11060
3	0.7328	0.6805	0.03702
4	0.7158	0.6983	0.01234
5	0.7100	0.7042	0.00412
6	0.7081	0.7061	0.00137
7	0.7074	0.7068	0.00046
8	0.7072	0.7070	0.00015

every number above is computed by the slide at compile time (chalkdust `la.matvec` + `la.normalize`) — converging to $(0.7071, 0.7071) = (1, 1)/\sqrt{2}$

The error, on a log axis

V VISUAL



A straight line on a log axis = **geometric** decay: the error divides by almost exactly 3 every step. Where does the 3 come from? Time for this lecture's one real proof.

⭐ Why it converges · step 1: change glasses

D DERIVATION

Write the start vector in the **eigen-basis** (we can — eigenvectors span the plane):

$$\boldsymbol{x}_0 = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 \qquad \boldsymbol{v}_1\colon \lambda_1 = 3; \quad \boldsymbol{v}_2\colon \lambda_2 = 1$$

For our run: $\boldsymbol{x}_0 = (1, 0)$ has $c_1 = c_2 = 1/\sqrt{2} \approx 0.7071$ — a half-and-half mix of the two eigen-directions.

This is THE move of the lecture: to understand what a matrix does to **any** vector, first express that vector in the matrix's own directions.

⭐ Step 2: apply A, k times

D DERIVATION

On eigenvectors, applying A is just multiplying by λ — so on the mix, term by term:

$$A\boldsymbol{x}_0 = c_1\lambda_1\boldsymbol{v}_1 + c_2\lambda_2\boldsymbol{v}_2$$

$$A^k\boldsymbol{x}_0 = c_1\lambda_1^k\boldsymbol{v}_1 + c_2\lambda_2^k\boldsymbol{v}_2 \quad = \quad c_1\,3^k\boldsymbol{v}_1 + c_2\,1^k\boldsymbol{v}_2$$

No matrix multiplication survived. The entire history of k applications is **two scalar powers** — the $\boldsymbol{v}_1$ ingredient grows like 3^k, the $\boldsymbol{v}_2$ ingredient like 1^k.

⭐ Step 3: factor out the winner

D DERIVATION

Pull the dominant growth λ_1^k out front:

$$A^k \boldsymbol{x}_0 = \lambda_1^k \left[c_1 \boldsymbol{v}_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k \boldsymbol{v}_2 \right]$$

Normalization kills the prefactor λ_1^k. Inside the bracket, since $|\lambda_2| < |\lambda_1|$:

$$\left(\frac{\lambda_2}{\lambda_1} \right)^k = \left(\frac{1}{3} \right)^k \to 0 \qquad \text{the } \boldsymbol{v}_2 \text{ contamination decays geometrically}$$

Power iteration converges to the top eigenvector, and the error shrinks by the factor $|\lambda_2/\lambda_1|$ per step — here $1/3$, exactly the slope the live plot measured.

The fine print (three honest conditions)

1. **An eigen-basis and a gap:** the derivation assumes the start can be expanded in eigenvectors (as for our symmetric matrix) and $|\lambda_1| > |\lambda_2|$ strictly. The closer the ratio to 1, the slower the convergence.
2. **A foothold:** need $c_1 \neq 0$ — the start must contain *some* $\boldsymbol{v}_1$. Random starts do with probability one under a continuous distribution. For this matrix, starting exactly at $(1, -1)$ stays on $\boldsymbol{v}_2$ even in float32 because the relevant operations are exactly representable.
3. **Normalize:** not for correctness — for survival. Un-normalized, $\|A^k \boldsymbol{x}\| \sim 3^k$ overflows float32 near $k \approx 81$.

The code: six honest lines

```
A = np.array([[2., 1.], [1., 2.]])
x = np.array([1., 0.])
for k in range(8):
    x = A @ x                    # multiply: let A pull
    x = x / np.linalg.norm(x)    # normalize: keep the direction
print(x)                         # [0.70718 0.70703]  -> v1
```

```
>>> np.linalg.eig(A).eigenvectors[:, 0]     # the library agrees
array([0.70710678, 0.70710678])
```

Six lines, no characteristic polynomial. The method can scale to huge **sparse or implicit** matrices when a matrix-vector product is cheap; a dense billion-row matrix would still be infeasible to store.

Checkpoint: power-iteration rate

Q QUESTION

B has eigenvalues $\lambda_1 = 4$ and $\lambda_2 = 2$. Per step of power iteration, the error shrinks by roughly a factor of…

A. $1/4$

B. $1/2$

C. $1/16$

D. it doesn't shrink — $\lambda_1 > 1$ makes it grow

Answer: the eigenvalue ratio sets the rate

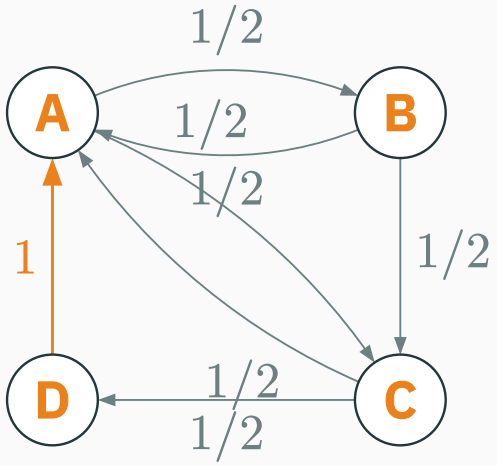
A ANSWER

Correct: B — $\times 1/2$ per step

Why: The contamination decays like $(\lambda_2/\lambda_1)^k = (2/4)^k$. The absolute sizes of the λ's are irrelevant after normalization — only the **ratio** matters. (D confuses growth of **length**, which normalization discards, with drift of **direction**.)

PageRank, almost answered

Back to the four pages — votes, made precise



Each page splits **one unit of importance** equally over its outgoing links:

- A and B and C each cast two half-votes;
- D links only to A — **one wholehearted vote**.

So importance flows: $r_A^{\text{new}} = \frac{1}{2} r_B + \frac{1}{2} r_C + r_D$, and likewise for the others.

The voting rule is a matrix

Stack the four flow equations; column j holds page j's outgoing vote shares:

$$\boldsymbol{r}^{\text{new}} = M\boldsymbol{r}, \qquad M = \begin{pmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 \end{pmatrix}$$

"Self-consistent ranking" now has an exact meaning — a vector the voting rule **reproduces**:

$\boldsymbol{r} = M\boldsymbol{r}$ i.e. an eigenvector of M with $\lambda = 1$

PageRank is an eigenvector. Ranking the web = power iteration on the link matrix.

Power-iterate the web — computed in this slide

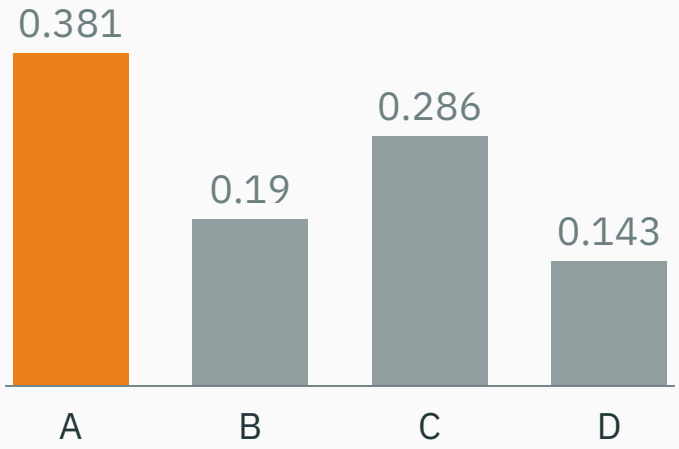
Start fair — a quarter each — and let the votes flow:

k	A	B	C	D
0	0.250	0.250	0.250	0.250
1	0.500	0.125	0.250	0.125
2	0.313	0.250	0.313	0.125
3	0.406	0.156	0.281	0.156
4	0.375	0.203	0.281	0.141
5	0.383	0.188	0.289	0.141
8	0.380	0.191	0.286	0.143
12	0.381	0.190	0.286	0.143

no normalization needed — each column of M sums to 1, so the total importance stays exactly 1.00.
Suspicious? Hold that thought.

Power iteration on the four-page graph

V VISUAL



- **A wins**: it collects votes from everyone — including D's undivided one.
- The exact limit is rational: $\boldsymbol{r} = (8, 4, 6, 3)/21$. Check the first row by hand: $\frac{1}{2} \cdot \frac{4}{21} + \frac{1}{2} \cdot \frac{6}{21} + \frac{3}{21} = \frac{8}{21}$ ✓ — the ranking votes for itself.

Did you see the wobble?

Look back at column A: 0.250 → 0.500 → 0.313 → 0.406 → 0.375 → … — it **overshoots and rings** like a struck bell, rather than sliding in monotonically.

Our convergence theory says the error decays by $|\lambda_2/\lambda_1|$. For this M the subdominant eigenvalues are $-\frac{1}{2}$ and a **complex pair** $-\frac{1}{4} \pm 0.43i$…

A complex-conjugate pair represents a rotation combined with scaling, so the error spirals toward zero. Its magnitude decreases by $|\lambda_2/\lambda_1| = 1/2$ per step.

One slide of honesty: this matrix is a Markov matrix

M's columns are nonnegative and sum to 1 — a **stochastic (Markov) matrix**.
Equivalent view: a "random surfer" clicks a uniformly random outgoing link forever; r_j becomes the fraction of time spent on page j.

- Why did $\lambda = 1$ exist at all? Rows of $M^\top$ sum to 1, so $M^\top \mathbf{1} = \mathbf{1}$ — and M, $M^\top$ share eigenvalues (same characteristic polynomial). **Every** Markov matrix has $\lambda = 1$.
- Our little web was safe: no dead-end pages, no isolated clique, so power iteration converged to a **unique** answer.

Real link graphs contain pages with no outlinks and closed communicating classes. PageRank adds a 0.15 teleport probability; L25 studies the Markov-chain conditions that make the stationary ranking unique.

Symmetric matrices: the nicest case

Our matrix was secretly the nicest kind

$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ equals its own transpose: $A = A^\top$ — **symmetric**. Now look at its eigenvectors:

$$\boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \boldsymbol{w} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \qquad \boldsymbol{v} \cdot \boldsymbol{w} = 1 - 1 = 0 \quad \text{— orthogonal!}$$

Perpendicular eigen-directions, real eigenvalues, no drama. Coincidence?

Not a coincidence. For symmetric matrices, all of it is guaranteed.

The spectral theorem (stated, savored, not proved)

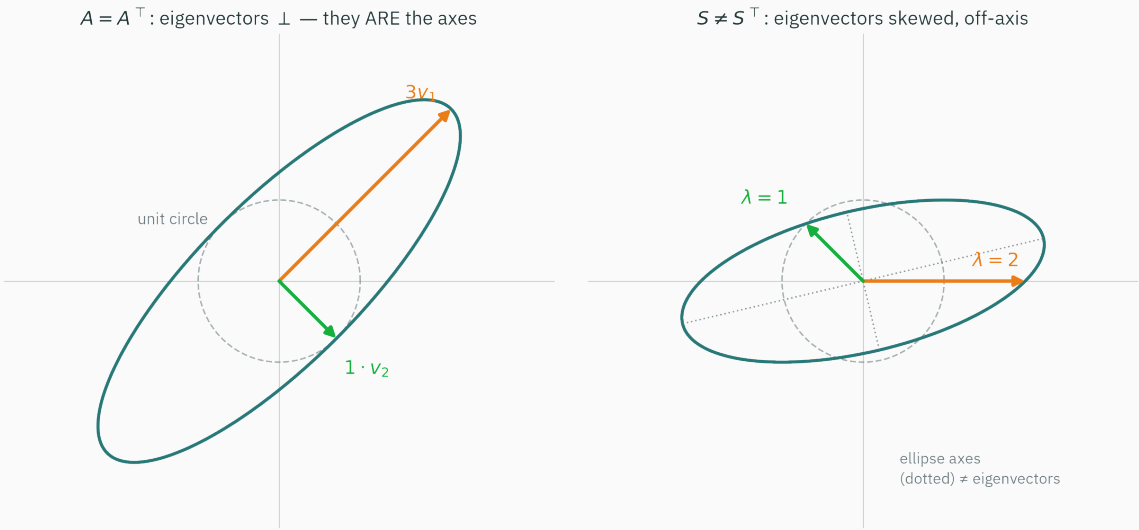
Theorem. Every real symmetric matrix $A = A^\top$ has:

1. all eigenvalues **real** — no rotation pairs can hide in it,
2. an **orthonormal eigenbasis** can be chosen — a perpendicular grid, including within repeated-eigenvalue subspaces,
3. hence the clean factorization $A = Q\Lambda Q^\top$, with Q's columns orthonormal ($Q^{-1} = Q^\top$ — inverting it is free).

Stated with a straight face, used forever, proved in MML §4.2 for the curious — the proof is not our derivation today. What you must own is the **picture**, next slide.

The picture: symmetric matrices act along a perpendicular grid

V VISUAL



symmetric = pure stretch along a perpendicular grid: eigenvectors **are** the ellipse axes — non-symmetric eigenvectors miss them

Planted for L13: a Gaussian data cloud's covariance matrix is symmetric — its eigenvectors will be the **axes of the data ellipse**, its eigenvalues the spreads.

The slide double-checks the theorem

Chalkdust's symmetric eigensolver (`la.eig-sym`, Jacobi rotations), run on A at compile time:

$$\lambda = (3, 1) \qquad \boldsymbol{q}_1 = \begin{pmatrix} 0.7071 \\ 0.7071 \end{pmatrix} \quad \boldsymbol{q}_2 = \begin{pmatrix} 0.7071 \\ -0.7071 \end{pmatrix}$$

$\boldsymbol{q}_1 \cdot \boldsymbol{q}_2 = 0.0000$ — orthogonal ✓

The reported value is zero to machine precision, the appropriate numerical interpretation after L2.

Real λ's, perpendicular $\boldsymbol{q}$'s — the theorem, holding up in silicon.

Why ML is wall-to-wall symmetric matrices

Symmetric matrices are not a corner case — ML manufactures them:

- **Covariance matrices** (L13) — the geometry of every data cloud;
- $X^\top X$ — at the heart of least squares (L4′s preview) and of PCA (L6);
- **Hessians** (L17–L18) — their eigenvalues determine curvature and stable learning-rate ranges.

Whenever ML hands you a matrix that is symmetric, the spectral theorem hands you back a perpendicular grid of dials. That guarantee powers L6, L13, and L17.

The take-home

Lecture 5 — summary

- **Eigen-pair:** $Av = \lambda v$ — for a real eigenpair, v keeps its line and λ sets scaling and possible sign reversal. A complex-conjugate pair describes a real two-dimensional invariant plane.
- **2×2 by hand:** solve $\det(A - \lambda I) = 0$; audit with trace $= \sum \lambda_i$, det $= \prod \lambda_i$.
- **Diagonalize:** $A = PDP^{-1}$ — in the eigen-basis A is pure dials, so $A^k = PD^kP^{-1}$.
- **Power iteration:** multiply-normalize → dominant eigenvector; error $\times\ |\lambda_2/\lambda_1|$ per step. PageRank uses the eigenvector with $\lambda = 1$ (L25).
- **Spectral theorem:** symmetric $\Rightarrow$ real λ's, orthogonal eigenvectors, $A = Q\Lambda Q^\top$ (→ L6, L13).

Before L6 — MML §4.1–4.2 · replay the Setosa explorable · 3Blue1Brown, *Essence of Linear Algebra*, ch. 14 — 17 minutes, gold.

Practice problems

Try on paper; then check yourself with `np.linalg.eig` and your six-line power iteration.

1. Eigenvalues and eigenvectors of $\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, by hand. Verify trace and det.
2. For $\begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$: predict the eigenvalues' **signs** from the det, then compute.
3. Two steps of power iteration on our A from $\boldsymbol{x}_0 = (0, 1)$. Why the same limit as from $(1, 0)$?
4. For the Markov matrix $\begin{pmatrix} 0.9 & 0.5 \\ 0.1 & 0.5 \end{pmatrix}$: find the $\lambda = 1$ eigenvector — you have just computed a stationary distribution (L25).
5. Why does rotation (by anything but $0°$ or $180°$) have no real eigenvectors? Picture first, then the discriminant.
6. Power-iterate A from exactly $\boldsymbol{x}_0 = (1, -1)$: what does theory predict? Why does float32 also remain on that direction for this particular integer matrix?

⭐⭐⭐ When diagonalization fails

★ OPTIONAL

The shear $J = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has characteristic polynomial $(1-\lambda)^2$: eigenvalue $\lambda = 1$, **twice**. But hunt for eigenvectors:

$$(J - I)\boldsymbol{v} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}\boldsymbol{v} = 0 \quad \Rightarrow \quad \text{only } \boldsymbol{v} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

One direction for a 2×2 matrix — not enough to build P. J is **defective**: it cannot be diagonalized, so the eigen-basis formula $A = PDP^{-1}$ and the power proof built from it do not apply. The eigenvalue equation and characteristic polynomial still do.

Two consolations. Symmetric matrices are **never** defective (spectral theorem — one more reason ML loves them). And next lecture's SVD factors **every** matrix, defective or not, square or not.

Eigenvectors are the directions a matrix can’t turn.

Next: **SVD & PCA** — what about non-square matrices? Every matrix gets a version of this.